# Exercise 7.1: Proving Identity with Kerberos

**Setup walkthrough**

You will extend the NordTech network from exercise 6.1. The same three machines, the same users (maria, jens, lars) and the same LDAP directory stay in place. You add a **Kerberos ticket server** (the KDC), get a ticket as Maria, and use it to log in to another machine **without typing the password**. Parts 0 to 4 are the exercise. Parts 5, 6 and 7 are **optional**: break it on purpose, offboard Jens, and use Kerberos for the desktop login.

> **Before you start:** take a VM snapshot of every machine. Part 1 changes the server, and a mistake can be hard to undo.

> **What you need from exercise 6.1:** the LDAP server with maria, jens and lars, and both desktops with working SSSD (you can log in with `su - maria`). If you changed Jens' shell to `nologin` in step 4.2, you will change it back if you do the optional Part 6.

---

## 0. The plan

| Machine | Role in 6.1 | Role in 7.1 |
|---|---|---|
| `ldap.nordtech.dk` | LDAP server | LDAP server **and KDC** (also reachable as `kdc.nordtech.dk`) |
| `desktop1` | LDAP client | Kerberos **client**: Maria gets a ticket here |
| `desktop2` | LDAP client | Kerberos **service**: SSH accepts tickets here |

**Values used in this guide.** If you change them, change them everywhere.

| Item | Value |
|---|---|
| Realm | `NORDTECH.DK` (a Kerberos realm is written in capital letters) |
| KDC name | `kdc.nordtech.dk` |
| Server IP | `192.168.X.X` *(the same IP as the LDAP server)* |
| Desktop 1 IP | `192.168.Y.Y` *(replace with the real IP)* |
| Desktop 2 IP | `192.168.Z.Z` *(replace with the real IP)* |

**Principals** (the names Kerberos knows):

| Principal | What it is |
|---|---|
| `maria@NORDTECH.DK` | Maria, for daily work |
| `jens@NORDTECH.DK` | Jens, the contractor |
| `maria/admin@NORDTECH.DK` | Maria's **administration** account, used only to manage Kerberos |
| `host/desktop2.nordtech.dk@NORDTECH.DK` | The machine desktop2, as a service (SSH uses it) |

Kerberos has its own list of users. **Maria exists twice: once in LDAP and once in Kerberos.** In this lab they have the same name and the same password, so it feels like one. They are two separate systems, and the LDAP server does not know the Kerberos password (or the other way around). Part 4 shows how they work together, and which of the two answers which question.

> **Why the KDC is on the LDAP server in this lab, and why that is bad design.** To save one VM, the KDC runs on the same machine as the LDAP server. That is fine for a lab and a bad idea in real life:
>
> - **One break-in gives the attacker everything.** The LDAP server holds *who the users are*. The KDC holds *the secret key of every user and every service*. Together, a single compromise gives the attacker the identities **and** the means to prove them.
> - **The KDC should be small.** It should run almost nothing else, with few logins and few open ports. This machine also runs slapd and the LAM web interface (Apache and PHP). Every extra service is one more way in.
> - **Least privilege.** The people who manage users in LDAP should not automatically be root on the machine that holds every key.
> - **One failure stops both.** If this machine is down, nobody can look up users **and** nobody can get tickets.
>
> In real life the KDC runs on its own, hardened machine, preferably with a second KDC as a backup. Clients only know the *name* `kdc.nordtech.dk`, so moving the KDC later means changing one IP address, not reconfiguring every client.

## 0.1 Names and clocks (all three machines)

Kerberos is picky about two things: **names** and **time**. A service ticket is issued for a *name*, so every machine must use the same full names, and every machine must be able to find the others by those names.

**Step A: find the three IP addresses.** On each machine run:

```
ip a
```

and write down the `inet` address of the main network interface (not `127.0.0.1`, which belongs to `lo`). Use these as `X`, `Y` and `Z` below.

**Step B: give each machine its full name.** Run **one** command on **each** machine (the server already has its name from exercise 6.1):

| Run this on... | Command |
|---|---|
| the **server** | already done in 6.1 (`ldap.nordtech.dk`) |
| **desktop1** | `sudo hostnamectl set-hostname desktop1.nordtech.dk` |
| **desktop2** | `sudo hostnamectl set-hostname desktop2.nordtech.dk` |

**Step C: edit `/etc/hosts` on all three machines.** Open the file on **each** machine:

```
sudo nano /etc/hosts
```

Your file already has a few lines from the installation and from exercise 6.1. You change it in **four small steps**. Do the same four steps on all three machines.

1. **Keep** the line `127.0.0.1  localhost`, and leave the IPv6 lines at the bottom (`::1 ...`, `fe00::0 ...`) alone.
2. **Comment out** any line that starts with `127.0.1.1` by putting a `#` in front of it (if there is no such line, skip this step). Ubuntu adds a line like `127.0.1.1  laura-desktop` that points the machine's own name to the loopback address. Kerberos looks up the machine's name to find out which service it is, and it would then get `127.0.1.1` instead of the real IP address, and fail.
3. **Add `kdc.nordtech.dk  kdc` to the end of the `ldap.nordtech.dk` line** that you made in exercise 6.1. You are not adding a new line. The server is both LDAP server and KDC, so one IP address gets both names.
4. **Add two new lines** for the desktops, one for `desktop1` and one for `desktop2`, each with its own IP address. Every machine must be able to find both desktops by their full names, so add both lines on all three machines, also on the desktop you are sitting at.

Here is an example. **Before** (desktop1, after exercise 6.1):

```
127.0.0.1       localhost
127.0.1.1       laura-desktop
192.168.X.X     ldap.nordtech.dk   ldap
```

**After** (the same on **all three machines**; the IPv6 lines are not shown):

```
127.0.0.1       localhost
#127.0.1.1      laura-desktop
192.168.X.X     ldap.nordtech.dk       ldap      kdc.nordtech.dk   kdc
192.168.Y.Y     desktop1.nordtech.dk   desktop1
192.168.Z.Z     desktop2.nordtech.dk   desktop2
```

What each name is used for:

| Name | Used for |
|---|---|
| `ldap.nordtech.dk` | the LDAP server (unchanged from 6.1) |
| `kdc.nordtech.dk` | the Kerberos ticket server. In this lab it is the **same machine** as `ldap.nordtech.dk`, so both names have the same IP |
| `desktop1.nordtech.dk` | where Maria gets her ticket |
| `desktop2.nordtech.dk` | the machine Maria logs in to with the ticket. You must type this **full name** in `ssh` later |

**Step D: check it.** Run on **each** machine, and compare with the table:

```
hostname -f
getent hosts kdc.nordtech.dk
getent hosts desktop2.nordtech.dk
```

| Run on... | `hostname -f` should print |
|---|---|
| server | `ldap.nordtech.dk` |

| Run on... | `hostname -f` should print |
|---|---|
| desktop1 | `desktop1.nordtech.dk` |
| desktop2 | `desktop2.nordtech.dk` |

On every machine, `getent hosts kdc.nordtech.dk` must show the server's IP (X), and `getent hosts desktop2.nordtech.dk` must show desktop2's IP (Z). If a name is missing or shows `127.0.1.1`, fix `/etc/hosts` before going on.

**Time.** Kerberos also refuses tickets if the clocks differ by more than **5 minutes**. Check on all three machines:

```
date -u
timedatectl
```

`date -u` prints the time in UTC. It must be the same on all three machines (within a few seconds). Kerberos works in UTC, so a machine that **shows** a different local time is fine as long as `date -u` matches. For example, the server may be set to the time zone UTC while the desktops show Danish time, which looks like one hour of difference but is the same moment. If you want them to look alike, set the time zone on the server: `sudo timedatectl set-timezone Europe/Copenhagen`.

**If `date -u` is different on the machines**

A difference of a few seconds is fine. Kerberos only refuses tickets when the clocks differ by more than 5 minutes. If the difference is bigger, fix it in this order, and check with `date -u` after each step:

1. **Let the machine fix itself.** Look at the line `System clock synchronized` in `timedatectl`. If it says `no`, run:

   ```
   sudo timedatectl set-ntp true
   ```

   Wait a minute, then check `date -u` again. This needs internet access in the VM.

2. **If it is still different, copy the server's time to the desktops.** On the **server**, print the time in UTC:

   ```
   date -u +"%Y-%m-%d %H:%M:%S"
   ```

   On **each desktop**, turn off automatic sync (otherwise it overwrites your change), and set the time you just read. Replace the example with the server's time:

   ```
   sudo timedatectl set-ntp false
   sudo date -u -s "2026-10-28 09:14:30"
   ```

   A few seconds of difference from typing is fine.

3. **Check again** on all three machines with `date -u`. They should now be within a few seconds.

Many VM programs can also keep the guest clock in sync with your computer (look for *time synchronization* in the VM settings). That is the easiest permanent fix.

---

# Part 1: The KDC

All commands in Part 1 run on the **server** (`ldap.nordtech.dk`).

## 1.1 Install the KDC

```
sudo apt update
sudo apt install krb5-kdc krb5-admin-server krb5-config
```

The installer asks three questions:

| Question | Answer |
|---|---|
| Default Kerberos version 5 realm | `NORDTECH.DK` |
| Kerberos servers for your realm | `kdc.nordtech.dk` |
| Administrative server for your Kerberos realm | `kdc.nordtech.dk` |

## 1.2 Create the realm

```
sudo krb5_newrealm
```

It asks for a **master password** twice. This protects the KDC database. Choose a long one and write it down for the lab; it is only needed if the database is restored.

When it finishes, check that both services run:

```
systemctl is-active krb5-kdc krb5-admin-server
```

Both should say `active`. The realm now exists, but it is empty: no users, no services.

## 1.3 Allow administration

The administration account `maria/admin` must be allowed to manage the database. Look at the access file:

```
sudo cat /etc/krb5kdc/kadm5.acl
```

It should contain this line (it means: any principal ending in `/admin` may do everything). If it does not, add it:

```
*/admin   *
```

Restart the admin service so it reads the file:

```
sudo systemctl restart krb5-admin-server
```

### 1.4 Create the principals

`kadmin.local` is the administration tool that runs **on the KDC itself** as root, and needs no password. You use it here only because `maria/admin` does not exist yet. From now on, administration is done as `maria/admin` with `kadmin -p maria/admin`.

```
sudo kadmin.local
```

At the `kadmin.local:` prompt:

```
addprinc maria
addprinc jens
addprinc maria/admin
listprincs
quit
```

Each `addprinc` asks for a password twice. In the lab, use the same password you gave the user in LDAP. For `maria/admin`, choose a **different, stronger** password. It is the key to the whole realm.

`listprincs` should list your three new principals, together with a handful of system ones (`krbtgt/NORDTECH.DK` is the ticket-granting key, the secret that makes tickets valid).

> **Why `maria` and `maria/admin`?** Same idea as in lesson 3: Maria works every day as `maria`, with no extra rights. Only when she must manage Kerberos does she use `maria/admin`. If someone steals her daily password, they cannot change the database.

---

## Part 2: Log in once on desktop1

All commands in Part 2 run on **desktop1**.

### 2.1 Install the client tools

```
sudo apt update
sudo apt install krb5-user
```

Answer the same three questions as before (`NORDTECH.DK`, `kdc.nordtech.dk`, `kdc.nordtech.dk`). The installer writes your answers into `/etc/krb5.conf`. Look at the file:

```
cat /etc/krb5.conf
```

The file is long, because it also contains a list of example realms (MIT, Stanford, Toronto and others). **Ignore the examples** and look for these lines under the three headings:

```
[libdefaults]
    default_realm = NORDTECH.DK

[realms]
    NORDTECH.DK = {
        kdc = kdc.nordtech.dk
        admin_server = kdc.nordtech.dk
    }

[domain_realm]
    .nordtech.dk = NORDTECH.DK
    nordtech.dk = NORDTECH.DK
```

The installer normally writes the first two parts for you. The two lines under `[domain_realm]` are usually missing. **Add any line that is not there**, directly under its heading:

```
sudo nano /etc/krb5.conf
```

Use `Ctrl+W` to search for a heading such as `[domain_realm]`. Save with `Ctrl+O`, `Enter`, and exit with `Ctrl+X`.

`[domain_realm]` tells Kerberos which realm a machine name belongs to.

## 2.2 Get a ticket

Log in as Maria. She is an LDAP user, so this works through SSSD, as in 6.1:

```
su - maria
```

Now ask the KDC for your first ticket (the **ticket-granting ticket**, TGT):

```
kinit
```

Enter Maria's Kerberos password. Nothing is printed when it works. Look at what you got:

```
klist
```

You should see one ticket, for `krbtgt/NORDTECH.DK@NORDTECH.DK`, with a start time and an **expiry time** (about 10 hours later). That ticket is Maria's wristband: proof that she has logged in once.

Throw it away and see what changes:

```
kdestroy
klist
```

`klist` now says there is no ticket cache. Get a new ticket with `kinit` again before you continue.

> **Where did the password go?** It was typed into `kinit` and used on desktop1 to make a key. It was never sent over the network. The KDC stores the same key, not the password. Two things to remember: the stored key is almost as valuable as the password (so the KDC is a crown jewel), and a recorded login request can be guessed against offline, so a weak password is still weak. Strong passwords (lesson 5) still matter.

---

# Part 3: Use the service on desktop2

In this part Maria logs in to desktop2 over SSH **without typing a password**. SSH is only the example service here; the next lecture is about SSH itself.

## 3.1 Install and prepare desktop2

On **desktop2**:

```
sudo apt update
sudo apt install krb5-user openssh-server
```

Answer the three Kerberos questions as before. Then do the same as in step 2.1: look at `/etc/krb5.conf` with `cat`, and add any of the lines that are missing.

## 3.2 Give desktop2 a key (the keytab)

For Maria to trust desktop2, and for desktop2 to trust the ticket, desktop2 needs a **service principal** with a secret key that only the KDC and desktop2 know. The key is stored in a file called a **keytab**.

On **desktop2**, start the administration tool over the network as `maria/admin`:

```
sudo kadmin -p maria/admin
```

At the `kadmin:` prompt:

```
addprinc -randkey host/desktop2.nordtech.dk
ktadd host/desktop2.nordtech.dk
quit
```

`-randkey` gives the service a random key (no one needs to type it). `ktadd` writes the key into `/etc/krb5.keytab` on desktop2. Check:

```
sudo klist -k
```

You should see `host/desktop2.nordtech.dk@NORDTECH.DK`. The keytab is a secret as valuable as a password. Look at who can read it:

```
ls -l /etc/krb5.keytab
```

It must be readable by root only. (Run `ktadd` only once for a service; each run creates a new key and makes the old keytab useless.)

## 3.3 Let SSH accept tickets

On **desktop2**, create a small config file:

```
sudo nano /etc/ssh/sshd_config.d/kerberos.conf
```

with this content:

```
GSSAPIAuthentication yes
```

(GSSAPI is the name of the interface that SSH uses to talk to Kerberos.) Restart SSH:

```
sudo systemctl restart ssh
```

## 3.4 The test: no password prompt

On **desktop1**, log in as Maria and get a ticket (if you still have one from step 2.2, `klist` shows it, and you can skip `kinit`):

```
su - maria
kinit
klist
```

`kinit` asks for Maria's Kerberos password, and `klist` should show the ticket for `krbtgt/NORDTECH.DK@NORDTECH.DK`. Now log in to desktop2:

```
ssh maria@desktop2.nordtech.dk
```

Use the **full name**, because the ticket is for `host/desktop2.nordtech.dk`. If it works, you are logged in without any password prompt. Look at what happened behind the scenes (still on desktop1, after you type `exit`):

```
klist
```

There are now **two** tickets: the TGT, and a new **service ticket** for `host/desktop2.nordtech.dk@NORDTECH.DK`. The client asked the KDC for it automatically when it needed it. That is the second and third step from the lecture.

To see Kerberos do its work, repeat the login with debug output:

```
ssh -v maria@desktop2.nordtech.dk 2>&1 | grep -i -E "gssapi|authenticat"
```

Look for `Authenticated to desktop2.nordtech.dk ... using "gssapi-with-mic"`.

Now compare with having no ticket:

```
kdestroy
ssh maria@desktop2.nordtech.dk
```

SSH now asks for a password. Without the ticket, it falls back to typing it. Press `Ctrl+C`, and run `kinit` again.

---

## Part 4: LDAP and Kerberos together

You have now used both systems. Which one answered which question?

Log in to desktop2 with the ticket (as in 3.4) and run:

```
id
getent passwd maria
```

Where did desktop2 learn that Maria has UID 10001, the home folder `/home/maria` and the group `sales`? From **LDAP**, through SSSD, exactly as in exercise 6.1. And what let her in without a password? The **Kerberos ticket**.

| Question | Who answered | How you saw it |
|---|---|---|
| Who is Maria? Which UID, home and groups? | LDAP | `id`, `getent passwd maria` |
| Is it really Maria? | Kerberos | `kinit`, `klist`, no password prompt |

This is the idea behind **Active Directory**: it is an LDAP directory and a Kerberos KDC that work together. On Linux, FreeIPA does the same.

> **Optional:** the desktop login itself can also use Kerberos, so that `su - maria` checks the password with the KDC and gives Maria a ticket automatically. That is the optional Part 7.

---

## Part 5: Break it on purpose (optional)

*This part is optional. It shows what happens when Kerberos is unhappy.*

### 5.1 Wrong clock

On **desktop1**, move the clock 10 minutes forward:

```
sudo timedatectl set-ntp false
sudo date -s "$(date -d '+10 minutes')"
su - maria
kdestroy
kinit
```

You should get an error like `Clock skew too great while getting initial credentials`. The KDC refuses, because a ticket with the wrong time looks like an old, replayed one.

Fix it:

```
sudo timedatectl set-ntp true
```

If that does not set the right time (no internet in the VM), set the clock back with `sudo date -s "$(date -d '-10 minutes')"`. Check with `timedatectl`, and run `kinit` again.

### 5.2 Expired ticket

On desktop1, as Maria, ask for a ticket that only lives for two minutes:

```
kinit -l 2m
klist
```

Wait three minutes, then:

```
klist
ssh maria@desktop2.nordtech.dk
```

The ticket is expired, and SSH asks for the password. Tickets expire on purpose: a stolen ticket is only useful for a limited time.

---

## Part 6: Offboard Jens (optional)

*This part is optional. It ties Kerberos back to the offboarding from exercise 6.1.*

Jens' contract has ended. In exercise 6.1 you changed his shell in LDAP. With Kerberos there is a second place to close.

**Preparation.** If you set Jens' shell to `nologin` or `/bin/false` in 6.1 step 4.2, change it back to `/bin/bash` first (in LAM, on the Unix tab). Then, on **both desktops**, clear the cache and restart the service:

```
sudo sss_cache -E
sudo systemctl restart sssd
getent passwd jens
```

`getent` must show `/bin/bash` at the end of the line. Otherwise the test below will not work.

### 6.1 Before: Jens can log in

On **desktop1**:

```
su - jens
kinit
ssh jens@desktop2.nordtech.dk
```

He gets in without a password prompt after `kinit`. Type `exit`, and **leave `jens`'s ticket in place** (do not run `kdestroy` yet).

## 6.2 Disable him in Kerberos

Open a **second terminal** on **desktop1**, as your own user (not as Jens, and no `sudo`). Administer Kerberos as `maria/admin`, not as root:

```
kadmin -p maria/admin
```

Enter the password of `maria/admin`. At the `kadmin:` prompt:

```
modprinc -allow_tix jens
getprinc jens
quit
```

`-allow_tix` means "allow tickets". With the minus, the KDC no longer issues tickets to Jens. In the output of `getprinc` you should see `DISALLOW_ALL_TIX` under *Attributes*.

This is least privilege at work: the administration account does the job, over the network, and nobody needs to be root on the KDC. Root and `kadmin.local` are only needed for the very first setup in step 1.4, when `maria/admin` does not exist yet.

## 6.3 After: what changed?

On **desktop1**, as Jens, **keep** the existing ticket, and try again:

```
ssh jens@desktop2.nordtech.dk
```

Write down what happens. A ticket he already holds is not cancelled; it is valid until it expires. Whether a **new service ticket** can still be issued from an old TGT depends on the Kerberos version, so look at what your system does.

Now ask for a **new** login:

```
kdestroy
kinit
```

You should see `Client's credentials have been revoked`. Jens can no longer get a ticket.

## 6.4 Close the other doors

Kerberos only controls tickets. Jens still has an **LDAP account**, so a normal password login on a desktop (through SSSD) may still work. A full offboarding does both:

1. Disable the principal in Kerberos (what you just did), or delete it with `delprinc jens`.
2. Disable the account in LDAP (as in 6.1 step 4.2: shell, groups, or removal).
3. Remember that tickets already issued live on until they expire. That is one reason tickets are short-lived, and why you can limit the lifetime for particular principals (for example with `modprinc -maxlife "1 hour" jens`).

Compare with lesson 5, where you had to find and lock Jens' account on every machine. Here you need two commands, in two central places.

---

# Part 7: Kerberos for the desktop login (optional)

*This part is optional. Until now the desktop login (`su - maria`) checked the password with the LDAP server, and you ran `kinit` yourself to get a ticket. Here you let the KDC check the password at login, so the login gives you the ticket.*

Do this on **desktop1 only**, so that desktop2 still works the old way for comparison. Take a VM snapshot first.

## 7.1 Make sure Jens has tickets

If you did Part 6, Jens' principal may still be disabled. Enable it again as `maria/admin` (`kadmin -p maria/admin`, then `modprinc +allow_tix jens`, then `quit`).

## 7.2 Change the SSSD configuration

On **desktop1**:

```
sudo cp /etc/sssd/sssd.conf /etc/sssd/sssd.conf.bak
sudo nano /etc/sssd/sssd.conf
```

In the `[domain/...]` section, keep `id_provider = ldap` (who the user is). Replace the line `auth_provider = ldap` with these three lines (the password check now goes to the KDC):

```
auth_provider = krb5
krb5_server = kdc.nordtech.dk
krb5_realm = NORDTECH.DK
```

Leave the other lines as they are. To get useful logs while you test, you can also add `debug_level = 6` in the same section. Restart SSSD and clear the cache:

```
sudo systemctl restart sssd
sudo sss_cache -E
```

## 7.3 Test it: the login gives you the ticket

Run this as **your own user** (for example `laura`), not as root:

```
su - maria
```

Do **not** use `sudo su - maria`. Root can become any user without a password, so no password is checked, and no ticket is made.

Type Maria's password. This is now her **Kerberos** password. Then look at the tickets, **without running `kinit`**:

```
klist
```

You should already have a ticket for `krbtgt/NORDTECH.DK@NORDTECH.DK`. The login created it. Now log in to desktop2:

```
ssh maria@desktop2.nordtech.dk
```

You should get in without a password prompt.

### 7.4 Test it: the KDC decides

Open a second terminal as your own user, and disable Jens as `maria/admin`:

```
kadmin -p maria/admin
```

At the `kadmin:` prompt: `modprinc -allow_tix jens`, then `quit`. Now try, as your own user:

```
su - jens
```

Type Jens' password. The login must **fail**, because the KDC no longer issues tickets to him. Enable him again (`modprinc +allow_tix jens`) and repeat. Now the login works.

Before the change, the password was checked against `userPassword` in the LDAP directory, and disabling the principal in Kerberos had no effect on `su - jens`. Now Kerberos decides.

### 7.5 What changed?

| Question | Before (LDAP login) | Now |
|---|---|---|
| Who checks the password at the desktop login? | the LDAP server | the KDC |
| Who tells the desktop about UID, home and groups? | LDAP | LDAP (unchanged) |
| Does the login give you a ticket? | no, you ran `kinit` | yes |
| The password for Maria | `userPassword` in LDAP | the Kerberos password |

This is the same split as in Part 4, now built into the login: LDAP says who you are, Kerberos proves it.

To go back, restore the old file: `sudo cp /etc/sssd/sssd.conf.bak /etc/sssd/sssd.conf`, then `sudo systemctl restart sssd`.

---

## Part 8: Questions to think about

**For everyone** (Parts 1 to 4)

1. Why is TLS not enough to protect a password? What does Kerberos do differently?
2. What is the difference between the TGT and the service ticket?

3. The KDC in this lab runs on the same machine as the LDAP server. Name three reasons why that is bad design, and say how you would set it up in real life.
4. Why do we have both `maria` and `maria/admin`? What does this have in common with `sudo` in lesson 3?

**If you did the optional Part 5**

5. What did you see when the clocks did not agree, and what did you see when the ticket had expired? Why is a short ticket lifetime a good thing?

**If you did the optional Part 6**

6. After you disabled Jens in Kerberos, what happened to the ticket he already had? Why is it not enough to disable only one of the two systems?

**If you did the optional Part 7**

7. After the change, which system checks Maria's password at the desktop login, and what is LDAP still used for? What happened when Jens' principal was disabled, and why did that not happen before?

---

## Troubleshooting

| Problem | What to check |
|---|---|
| `kinit`: *Cannot resolve network address for KDC* or *Cannot contact any KDC* | Check `/etc/hosts` for `kdc.nordtech.dk`, `ping kdc.nordtech.dk`, and that `krb5-kdc` runs on the server: `systemctl status krb5-kdc` |
| `kinit`: *Clock skew too great* | The clocks differ by more than 5 minutes. Fix with `timedatectl` (see 0.1 and 5.1) |
| `kinit`: *Client not found in Kerberos database* | The principal does not exist. On the server: `sudo kadmin.local -q listprincs`. Check the spelling and that you typed the realm in capital letters |
| `kinit`: *Preauthentication failed* | Wrong password. Remember that the Kerberos password is separate from the LDAP password |
| `kadmin` says *Operation requires "add" privilege* | `/etc/krb5kdc/kadm5.acl` is missing the line `*/admin *`, or you did not restart `krb5-admin-server` (step 1.3) |
| `kadmin` cannot connect | Check that `krb5-admin-server` is active on the server and that you use `kdc.nordtech.dk`, not `localhost` |
| `ssh` still asks for a password | Run `klist` (do you have a ticket?). Run `ssh -v` and look for `gssapi`. Use the **full name** `desktop2.nordtech.dk`. Check that `GSSAPIAuthentication yes` is set on desktop2 and that SSH was restarted |
| `ssh -v` says *Server not found in Kerberos database* | The name does not match the service principal. Use `desktop2.nordtech.dk`, and check that both desktops resolve that name to the right IP (0.1) |
| `ssh` fails on desktop2 with *Permission denied (gssapi-with-mic)* | The keytab is missing or unreadable (`sudo klist -k`, step 3.2). Or the user does not exist on desktop2: `getent passwd maria` |

| Problem | What to check |
| --- | --- |
| After `ktadd` the old keytab no longer works | Each `ktadd` creates a new key. Run it once per service, or copy the newest keytab |
| `krb5_newrealm` seems to hang | It needs random data. Wait a minute, or move the mouse and press keys in the VM |
| `su - maria` fails after Part 7 | Use the **Kerberos** password. Check the clock and that `kdc.nordtech.dk` resolves. Read `sudo journalctl -u sssd -n 30 --no-pager` and `sudo tail -n 30 /var/log/sssd/krb5_child.log` (the file appears with `debug_level`). To go back, restore `sssd.conf.bak` and restart `sssd` |
| `Client's credentials have been revoked` | Expected after Part 6. The principal has `-allow_tix`. Re-enable it as `maria/admin`: `kadmin -p maria/admin`, then `modprinc +allow_tix jens` |